Министерство образования Республики Беларусь

Учреждение образования

«Брестский Государственный технический университет»

Кафедра ИИТ

Лабораторная работа № 1

По дисциплине: «КиСБ»

Тема: «Генерирование равномерно распределённых псевдослучайных последовательностей»

Выполнила:
Студентка 4 курса
Группы АС-65
Рапин Е. Ю.
Проверила:
Хацкевич А. С.

Брест 2026

Цель: освоить программную реализацию и практическое применение алгоритмов генерации псевдослучайных последовательностей (РСП), методов их статистической оценки, а также вероятностных и детерминированных тестов простоты чисел с построением доказуемо простых чисел.

Ход работы

Часть 1:

1. Согласно варианту реализовать приложение, генерирующие псевдослучайную равномерно распределенную последовательность произвольной длины из заданного алфавита.

2. Подобрать параметры данного генератора таким образом, чтобы период последовательности имел максимальное значение.

3. Использую статистическое тестирование, проверить гипотезу о том, что сгенерированная последовательность действительно имеет равномерное распределение (см. приложение 2).

4. Отчет должен обязательно содержать несколько тестов, отображающих изменение статистических свойств последовательностей в зависимости от параметров генератора.

Вариант	Генератор	Размер алфавита	Дополнение
11	Квадратичный конгруэнтный генератор	N = 10	Рассмотреть следующие случаи: 1. Генератор Ковью: c = 0 2. c != 0

1. Реализация генератора.

Реализован класс QuadraticCongruentialGenerator, порождающий псевдослучайную последовательность по формуле:

$$x_{i+1} = (a \cdot x_i^2 + b \cdot x_i + c) \; mod \, N$$

Алфавит задан согласно варианту : N = 10, т.е. значения {0, 1, ..., 9}.

2. Подбор параметров с максимальным периодом.

Выполнен полный перебор всех допустимых комбинаций:

a ∈ {1, ..., N−1} (при a = 0 генератор вырождается);

b ∈ {0, ..., N−1};

x0 ∈ {0, ..., N−1};

для второго случая — дополнительно c ∈ {1, ..., N−1}.

Случай 1: генератор Ковью (c = 0)

```
Случай 1: Генератор Ковью (c = 0)
Лучшие параметры: a=5, b=2, c=0, x0=1
Период T = 4, предпериод tau = 0
Первые 30 элементов: [1, 7, 9, 3, 1, 7, 9, 3, 1, 7, 9, 3, 1, 7, 9, 3, 1, 7, 9, 3, 1, 7, 9, 3, 1, 7]
```

При c = 0 нуль всегда является неподвижной точкой (0 → 0), а остальные значения образуют короткие циклы. Максимально достижимый период для N = 10 при c = 0 равен 4 — это подтверждается полным перебором.

Случай 2: c ≠ 0

```
Случай 2: c ≠ 0
Лучшие параметры: a=5, b=6, c=1, x0=0
Период T = 10, предпериод tau = 0
Первые 30 элементов: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Период T = 10 = N — максимально возможный для алфавита мощности 10. Проверка вручную:

- x = 0: (5·0 + 6·0 + 1) mod 10 = 1
- x = 1: (5·1 + 6·1 + 1) mod 10 = 12 mod 10 = 2
- x = 2: (5·4 + 6·2 + 1) mod 10 = 33 mod 10 = 3
- …
- x = 9: (5·81 + 6·9 + 1) mod 10 = 460 mod 10 = 0

Формула действительно порождает полный цикл.

Параметры подобраны оптимально — в случае c ≠ 0 достигнут максимальный период T = N = 10.

3. Статистическая проверка равномерности.

Сгенерирована последовательность длины 1000. Применены четыре теста.

Случай 1: генератор Ковью (c = 0)

```
Хи-квадрат: 1500.0000, df=9, крит.=16.9190 → Не пройден
Частоты: {0: 0.0, 1: 0.25, 2: 0.0, 3: 0.25, 4: 0.0, 5: 0.0, 6: 0.0, 7: 0.25, 8: 0.0, 9: 0.25}
Число серий: 1000 (ожидается ≈ 900)
Автокорреляция (lag=1): -0.0008
```

Последовательность не является равномерно распределённой — используются только 4 из 10 значений алфавита. Это прямое следствие короткого периода T = 4.

Случай 2: c ≠ 0

```
Хи-квадрат: 0.0000, df=9, крит.=16.9190 → Пройден
Частоты: {0: 0.1, 1: 0.1, 2: 0.1, 3: 0.1, 4: 0.1, 5: 0.1, 6: 0.1, 7: 0.1, 8: 0.1, 9: 0.1}
Число серий: 1000 (ожидается ≈ 900)
Автокорреляция (lag=1): 0.4570
```

По частотам распределение идеально равномерное (каждое значение встречается ровно 100 раз из 1000). Однако автокорреляция 0.457 показывает, что соседние элементы сильно зависимы — последовательность строго монотонна 0, 1, 2, ... Это типично для детерминированного генератора с максимальным периодом: равномерность по частотам достигается, но непредсказуемость — нет.

Гипотеза о равномерном распределении принимается для случая c ≠ 0 (хи-квадрат = 0) и отвергается для случая c = 0 (хи-квадрат = 1500).

4. Влияние параметров на статистические свойства.

```
Сравнение статистических свойств при разных параметрах
  a   b   c  x0     T      chi2   прошёл
  5   2   0   1     4 1500.0000      нет
  3   5   0   2     1 9000.0000      нет
  7   3   1   4     1 8960.0600      нет
  2   3   2   1     2 4000.0000      нет
  4   1   3   2     6  666.6800      нет
  5   6   1   0    10    0.0000       да
```

Таблица демонстрирует монотонную зависимость χ^2 от периода T (чем больше период T, тем меньше χ^2). Это показывает, что:

- при c = 0 максимальный период мал (T = 4) → тест не проходит;
- при c ≠ 0 можно достичь T = N = 10 → тест проходит;
- параметры с малым периодом дают χ^2 в тысячи раз больше критического значения.

Даже при максимальном периоде квадратичный конгруэнтный генератор даёт линейно зависимую последовательность (высокая автокорреляция). Это принципиальное свойство конгруэнтных генераторов — они пригодны для моделирования, но не для криптографии без дополнительных преобразований.

Часть 2:

1. Реализовать приложение, позволяющее генерировать простое число по следующей схеме.

2. В отчете обязательно должны присутствовать тесты, подтверждающие правильность разработанных программ.

Вариант	Алгоритм
11	Полиномиальный тест распознавания простоты

1. Реализация схемы генерации простого числа.

Реализована функция generate_prime(bits), которая генерирует простое число заданной битовой длины строго по схеме из методички. Источник случайности — квадратичный конгруэнтный генератор из Части 1 (класс QCGBitSource на базе QuadraticCongruentialGenerator).

2. Тесты корректности.

Набор подобран так, чтобы покрыть все «сложные» случаи:

1. Малые простые и составные (2, 3, 4, 5, 17) — проверяют граничные случаи алгоритма.
2. Полные квадраты ($25 = 5^2$, $121 = 11^2$) — проверяют шаг 1 алгоритма AKS.
3. Полупростое ($341 = 11 \cdot 31$) — классический пример псевдопростого по основанию 2. Тест Ферма на нём даёт «не известно», но AKS обязан отвергнуть.
4. Числа Кармайкла (561, 1105, 1729) — составные числа, которые «обманывают» тест Ферма при всех основаниях a, взаимно простых с n. AKS — детерминированный тест и обязан их отвергать.

```
Тесты корректности на известных числах
  n =       2  →  простое     [OK]
  n =       3  →  простое     [OK]
  n =       4  →  составное   [OK]
  n =       5  →  простое     [OK]
  n =      17  →  простое     [OK]
  n =      25  →  составное   [OK]
  n =      97  →  простое     [OK]
  n =     100  →  составное   [OK]
  n =     101  →  простое     [OK]
  n =     121  →  составное   [OK]
  n =     341  →  составное   [OK]
  n =     561  →  составное   [OK]
  n =    1105  →  составное   [OK]
  n =    1729  →  составное   [OK]

Все тесты: Пройдены
```

Схема генерации работает — оба числа действительно простые и действительно имеют заданную битовую длину. Источник битов на ККГ корректен — биты, извлечённые из ККГ, дают валидных кандидатов, среди которых находится простое число. AKS работает как фильтр — не пропускает составные числа.

```
   8-битовое простое:        131  (проверка AKS: простое)
  12-битовое простое:       2111  (проверка AKS: простое)
```

Имеется набор из 50 подряд идущих составных чисел от 4 до 70. Все 50 чисел корректно распознаны как составные. Это подтверждает, что AKS не даёт ложноположительных результатов на этом диапазоне.

```
Проверка отвержения составных чисел
  Отвергнуто 50 из 50 составных чисел
```

Полиномиальный тест AKS — детерминированный, а не вероятностный. Он не требует нескольких повторений с разными a: одного прохода достаточно для строгого ответа. Поэтому в реализации aks_primality_test не добавлен внешний цикл пять раз с разными a. Более того, в AKS параметр a — не случайное основание, а перебираемый индекс от 1 до $\lfloor 2\sqrt{r}\log_2 n \rfloor$, и проверка идёт для всех a из этого диапазона.

Код программы:

```
import random
import math
from math import gcd, isqrt, log2, ceil
from collections import Counter

# ЧАСТЬ 1.

class QuadraticCongruentialGenerator:
    def __init__(self, a, b, c, x0, N=10):
        if N <= 0:
            raise ValueError("N должно быть > 0")
        self.a = a % N
        self.b = b % N
        self.c = c % N
        self.x0 = x0 % N
        self.N = N
        self.reset()
```

```
    def reset(self):
        self.x = self.x0
        self._history = [self.x]

    def next(self):
        # Один шаг генератора по формуле
        self.x = (self.a * self.x * self.x + self.b * self.x + self.c) % self.N
        self._history.append(self.x)
        return self.x

    def generate(self, length):
        # Генерация последовательности заданной длины
        self.reset()
        seq = [self.x]
        for _ in range(length - 1):
            seq.append(self.next())
        return seq

    def find_period(self):
        seen = {}
        self.reset()
        x = self.x
        i = 0
        while x not in seen:
            seen[x] = i
            x = self.next()
            i += 1
        tau = seen[x]
        T = i - tau
        return tau, T

def find_best_params(N=10, c=0):
    best = (None, None, None, -1, -1)
    for a in range(1, N):
        for b in range(N):
            for x0 in range(N):
                gen = QuadraticCongruentialGenerator(a, b, c, x0, N)
                tau, T = gen.find_period()
                if T > best[3]:
                    best = (a, b, x0, T, tau)
    return best

# Статистические тесты

def chi_square_test(seq, N=10):
    # Критерий хи-квадрат: проверка равномерности распределения
    n = len(seq)
    observed = Counter(seq)
    expected = n / N
    chi2 = sum((observed.get(k, 0) - expected) ** 2 / expected for k in range(N))
    df = N - 1
    critical = {
        1: 3.841, 2: 5.991, 3: 7.815, 4: 9.488, 5: 11.070,
        6: 12.592, 7: 14.067, 8: 15.507, 9: 16.919,
        10: 18.307
    }.get(df, 16.919)
    return chi2, df, critical, chi2 < critical

def frequency_test(seq, N=10):
    # Относительные частоты каждого значения алфавита
    n = len(seq)
    freq = Counter(seq)
    return {k: freq.get(k, 0) / n for k in range(N)}

def runs_test(seq, N=10):
    # Тест серий: число серий подряд идущих одинаковых значений
    n = len(seq)
    if n < 2:
        return 0, 0
    runs = 1
    for i in range(1, n):
        if seq[i] != seq[i - 1]:
            runs += 1
    expected = n * (1 - 1 / N)
```

```
    return runs, expected

def serial_correlation_test(seq, lag=1):
    # Выборочная автокорреляция с заданным лагом
    n = len(seq)
    if n <= lag:
        return 0.0
    mean = sum(seq) / n
    num = sum((seq[i] - mean) * (seq[i + lag] - mean) for i in range(n - lag))
    den = sum((x - mean) ** 2 for x in seq)
    return num / den if den != 0 else 0.0

def demo_qcg():
    print("\nЧасть 1")
    print("Квадратичный конгруэнтный генератор (N = 10)")

    N = 10

    print("\nСлучай 1: Генератор Ковью (c = 0)")
    a, b, x0, T, tau = find_best_params(N, c=0)
    print(f"Лучшие параметры: a={a}, b={b}, c=0, x0={x0}")
    print(f"Период T = {T}, предпериод tau = {tau}")

    gen = QuadraticCongruentialGenerator(a, b, 0, x0, N)
    seq = gen.generate(1000)
    print(f"Первые 30 элементов: {seq[:30]}")

    chi2, df, crit, passed = chi_square_test(seq, N)
    print(f"\nХи-квадрат: {chi2:.4f}, df={df}, крит.={crit:.4f} → "
          f"{'Пройден' if passed else 'Не пройден'}")
    freq = frequency_test(seq, N)
    print("Частоты:", {k: round(v, 4) for k, v in freq.items()})
    runs, expected_runs = runs_test(seq, N)
    print(f"Число серий: {runs} (ожидается ≈ {expected_runs:.0f})")
    print(f"Автокорреляция (lag=1): {serial_correlation_test(seq, 1):.4f}")

    print("\nСлучай 2: c ≠ 0")
    best = (None, None, None, None, -1, -1)
    for c in range(1, N):
        for a in range(1, N):
            for b in range(N):
                for x0 in range(N):
                    g = QuadraticCongruentialGenerator(a, b, c, x0, N)
                    t, T = g.find_period()
                    if T > best[4]:
                        best = (a, b, c, x0, T, t)
    a, b, c, x0, T, tau = best
    print(f"Лучшие параметры: a={a}, b={b}, c={c}, x0={x0}")
    print(f"Период T = {T}, предпериод tau = {tau}")

    gen = QuadraticCongruentialGenerator(a, b, c, x0, N)
    seq = gen.generate(1000)
    print(f"Первые 30 элементов: {seq[:30]}")

    chi2, df, crit, passed = chi_square_test(seq, N)
    print(f"\nХи-квадрат: {chi2:.4f}, df={df}, крит.={crit:.4f} → "
          f"{'Пройден' if passed else 'Не пройден'}")
    freq = frequency_test(seq, N)
    print("Частоты:", {k: round(v, 4) for k, v in freq.items()})
    runs, expected_runs = runs_test(seq, N)
    print(f"Число серий: {runs} (ожидается ≈ {expected_runs:.0f})")
    print(f"Автокорреляция (lag=1): {serial_correlation_test(seq, 1):.4f}")

    print("\nСравнение статистических свойств при разных параметрах")
    print(f"{'a':>3} {'b':>3} {'c':>3} {'x0':>3} {'T':>5} {'chi2':>9} {'прошёл':>8}")
    test_cases = [
        (5, 2, 0, 1), (3, 5, 0, 2), (7, 3, 1, 4),
        (2, 3, 2, 1), (4, 1, 3, 2), (5, 6, 1, 0),
    ]
    for (ta, tb, tc, tx) in test_cases:
        g = QuadraticCongruentialGenerator(ta, tb, tc, tx, N)
        t, T = g.find_period()
        s = g.generate(1000)
        chi2, _, _, passed = chi_square_test(s, N)
        print(f"{ta:>3} {tb:>3} {tc:>3} {tx:>3} {T:>5} {chi2:>9.4f} "
```

```
                f"{'да' if passed else 'нет':>8}")

# ЧАСТЬ 2.

def is_perfect_power(n: int) -> bool:
    # Проверка, является ли n совершенной степенью a^b (b >= 2)
    if n <= 1:
        return False
    max_b = int(log2(n)) + 1
    for b in range(2, max_b + 1):
        a = round(n ** (1 / b))
        if a ** b == n:
            return True
    return False

def is_prime_small(n: int) -> bool:
    # Проверка простоты для небольших чисел
    if n < 2:
        return False
    if n < 4:
        return True
    if n % 2 == 0:
        return False
    d = 3
    while d * d <= n:
        if n % d == 0:
            return False
        d += 2
    return True

def largest_prime_factor(m: int) -> int:
    # Наибольший простой делитель числа m
    largest = 1
    d = 2
    while d * d <= m:
        while m % d == 0:
            largest = d
            m //= d
        d += 1 if d == 2 else 2
    if m > 1:
        largest = m
    return largest

def mod_pow(base: int, exp: int, mod: int) -> int:
    return pow(base, exp, mod)

def poly_mod_mul(p, q, r, n):
    # Умножение полиномов по модулю (x^r - 1) с коэффициентами mod n
    res = [0] * r
    for i, pi in enumerate(p):
        if pi == 0:
            continue
        for j, qj in enumerate(q):
            if qj == 0:
                continue
            res[(i + j) % r] = (res[(i + j) % r] + pi * qj) % n
    return res

def poly_mod_pow(base_poly, exp, r, n):
    # Возведение полинома в степень exp по модулю (x^r - 1, n)
    result = [1] + [0] * (r - 1)
    base = base_poly[:]
    while exp > 0:
        if exp & 1:
            result = poly_mod_mul(result, base, r, n)
        base = poly_mod_mul(base, base, r, n)
        exp >>= 1
    return result

def check_poly_congruence(n: int, a: int, r: int) -> bool:
    # Проверка сравнения (x + a)^n ≡ x^n + a (mod n, x^r - 1)
    left = poly_mod_pow([a % n, 1] + [0] * (r - 2), n, r, n)
    right = [0] * r
    right[n % r] = (right[n % r] + 1) % n
    right[0] = (right[0] + a) % n
```

```
    return left == right

# Полиномиальный тест

def aks_primality_test(n: int) -> bool:
    if n <= 1:
        return False
    if n <= 3:
        return True
    if n % 2 == 0:
        return False

    # Шаг 1: проверка на совершенную степень
    if is_perfect_power(n):
        return False

    # Шаги 2–7: поиск подходящего r
    r = 2
    while r < n:
        if gcd(n, r) != 1:
            return False  # найден делитель n

        if is_prime_small(r):
            q = largest_prime_factor(r - 1)
            # Условие выхода из цикла: q > 4*sqrt(r)*log2(n) и n^((r-1)/q) != 1 (mod r)
            if q > 4 * math.sqrt(r) * log2(n):
                if mod_pow(n, (r - 1) // q, r) != 1:
                    break
        r += 1

    if r >= n:
        return True  # n простое

    # Шаги 9–10: проверка полиномиальных сравнений
    limit = int(2 * math.sqrt(r) * log2(n))

    if n - 1 <= limit:
        # Для очень маленьких n достаточно проверить НОД
        for a in range(1, n):
            if gcd(a, n) != 1:
                return False
        return True

    # Перебор всех a от 1 до limit
    for a in range(1, limit + 1):
        if not check_poly_congruence(n, a, r):
            return False

    return True

SMALL_PRIMES = [
    2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71,
    73, 79, 83, 89, 97, 101, 103, 107, 109, 113, 127, 131, 137, 139, 149, 151,
    157, 163, 167, 173, 179, 181, 191, 193, 197, 199, 211, 223, 227, 229, 233,
    239, 241, 251
]

def is_divisible_by_small_prime(n: int) -> bool:
    # Проверка делимости на малые простые (<= 256)
    for p in SMALL_PRIMES:
        if n == p:
            return False
        if n % p == 0:
            return True
    return False

class QCGBitSource:
    # Источник битов на основе ККГ из части 1

    def __init__(self, bits: int):
        self.bits = bits
        self.N = 1 << bits

        # Параметры ККГ, обеспечивающие большой период
        a = (1 << (bits - 1)) + 3
```

```
        b = (1 << (bits - 2)) + 1
        c = 1
        x0 = ((1 << bits) - 1) // 3 + 1

        self.gen = QuadraticCongruentialGenerator(a, b, c, x0, self.N)

    def getrandbits(self, bits: int) -> int:
        # Извлечение случайных битов из последовательных значений ККГ
        result = 0
        collected = 0
        while collected < bits:
            x = self.gen.next()
            take = min(self.bits, bits - collected)
            chunk = x & ((1 << take) - 1)
            result = (result << take) | chunk
            collected += take
        return result

def generate_prime(bits: int, max_attempts: int = 100000) -> int:
    # Генерация простого числа
    if bits < 2:
        raise ValueError("bits >= 2")

    qcg_source = QCGBitSource(bits)

    attempts = 0
    while attempts < max_attempts:
        attempts += 1
        # Шаг 1: случайное p-битовое число (биты из ККГ)
        n = qcg_source.getrandbits(bits)
        # Шаг 2: установка старшего и младшего битов
        n |= (1 << (bits - 1)) | 1
        # Шаг 3: отсев по малым простым
        if is_divisible_by_small_prime(n):
            continue
        # Шаг 4: полиномиальный тест простоты
        if aks_primality_test(n):
            return n
    raise RuntimeError("Не удалось сгенерировать простое число")

def demo_aks():
    print("\nЧасть 2")
    print("Полиномиальный тест распознавания простоты (AKS)")

    print("\nТесты корректности на известных числах")
    test_numbers = [
        (2, True), (3, True), (4, False), (5, True),
        (17, True), (25, False), (97, True), (100, False),
        (101, True), (121, False), (341, False),
        (561, False),
        (1105, False),
        (1729, False),
    ]
    all_ok = True
    for num, expected in test_numbers:
        result = aks_primality_test(num)
        status = "OK" if result == expected else "FAIL"
        if result != expected:
            all_ok = False
        print(f"  n = {num:7d}  →  {'простое' if result else 'составное':10s}  [{status}]")
    print(f"\nВсе тесты: {'Пройдены' if all_ok else 'Есть ошибки'}")

    print("\nГенерация простых чисел заданной битовой длины")
    print("(источник битов — квадратичный конгруэнтный генератор из части 1)")
    for bits in (8, 12):
        p = generate_prime(bits)
        check = aks_primality_test(p)
        print(f"  {bits:2d}-битовое простое: {p:12d}  "
              f"(проверка AKS: {'простое' if check else 'составное'})")

    print("\nПроверка отвержения составных чисел")
    composite_tests = [
        4, 6, 8, 9, 10, 12, 14, 15, 16, 18, 20, 21, 22, 24, 25, 26, 27, 28,
        30, 32, 33, 34, 35, 36, 38, 39, 40, 42, 44, 45, 46, 48, 49, 50,
        51, 52, 54, 55, 56, 57, 58, 60, 62, 63, 64, 65, 66, 68, 69, 70,
```

```
    ]
    rejected = sum(1 for x in composite_tests if not aks_primality_test(x))
    print(f"  Отвергнуто {rejected} из {len(composite_tests)} составных чисел")

if __name__ == "__main__":
    demo_qcg()   # Часть 1
    demo_aks()   # Часть 2
```

Вывод: в ходе работы освоены алгоритмы генерации равномерно распределенных псевдослучайных последовательностей на примере квадратичного конгруэнтного генератора. Практически опровергнута (случай 1) и подтверждена по частотам (случай 2) равномерность с помощью статистических тестов. Изученные принципы успешно применены для формирования кандидатов в алгоритме поиска простых чисел.